DHANUSH KUMAR R
192411215
DSA0120-OPP C++

CO 2 - ASSIGNMENT

Q1

Programiz
C++ Online Compiler

Install | Programiz PRO

main.cpp | Clear | Run

```cpp

#include <iostream>
#include <string>

struct Student {             cost
    std::string name;
    long long fee;
    char documents[1000];
};

void concessionByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
    std::cout << "  inside by-value    : " << fee << '\n';
}
void concessionByRef(long long& fee, int percent) {
    fee -= fee * percent / 100;
    std::cout << "  inside by-reference: " << fee << '\n';
}
void concessionByPtr(long long* fee, int percent) {
    if (!fee) return;
    *fee -= *fee * percent / 100;
    std::cout << "  inside by-pointer : " << *fee << '\n';
}
long long concessionReturn(long long fee, int percent) {
    return fee - fee * percent / 100;
}
long long readFee(const Student& s) { return s.fee; }

int main() {
    long long f1 = 45000, f2 = 45000, f3 = 45000;

    std::cout << "Predicted stored fee -> value: 45000,
```

Output

```
Predicted stored fee -> value: 45000, reference: 40500, pointer:
40500

  inside by-value    : 40500
stored fee after by-value     : 45000
  inside by-reference: 40500
stored fee after by-reference: 40500
  inside by-pointer : 40500
stored fee after by-pointer   : 40500

merit(10) then sports(15) via return value: 34425

sizeof(long long) = 8, sizeof(long long&) pointer-sized = 8,
sizeof(Student) = 1040
readFee(const Student&) = 45000
```

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

Programiz
C++ Online Compiler

Install | Programiz PRO

main.cpp | Clear | Run

```cpp

#include <iostream>
void change(int a, int& b, int* c)
{
    a += 10;
    b += 10; *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}

```

Output

```
1 12 13
1 100
16 13 6
```

Programiz
C++ Online Compiler

Install | Programiz PRO

main.cpp | Clear | Run

```cpp

#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent)
{

    fee -= fee * percent / 100;
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}

```

Output

```
after call by value : 45000
after call by reference: 40500
```

Q2

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> marks = {72, 88, 65, 91};
    marks.shrink_to_fit();
    int* before = &marks[0];
    std::cout << "capacity before : " << marks.capacity() << ", &marks[0] = " << before << '\n';

    marks.push_back(79);
    std::cout << "capacity after  : " << marks.capacity() << ", &marks[0] = " << &marks[0] << '\n';

    if (before != &marks[0])
        std::cout << "Storage MOVED: any reference/pointer taken earlier is now dangling.\n";
    else
        std::cout << "Storage did not move this time (capacity was enough).\n";


    std::size_t idx = 3;
    marks.push_back(60); marks.push_back(61); marks.push_back(62);
    marks[idx] += 2;
    std::cout << "marks[3] via index after growth: " << marks[idx] << '\n';
}
```

Output

```
capacity before : 4, &marks[0] = 0x3ae532b0
capacity after  : 8, &marks[0] = 0x3ae536e0
Storage MOVED: any reference/pointer taken earlier is now dangling.
marks[3] via index after growth: 93
```

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp
#include <iostream>

int& highest(int marks[], int n) {
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}
const int& highestConst(const int marks[], int n) {
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}
int highestCopy(const int marks[], int n) {
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}
void print(const char* label, const int m[], int n) {
    std::cout << label;
    for (int i = 0; i < n; ++i) std::cout << m[i] << ' ';
    std::cout << '\n';
}

int main() {
    int marks[] = {72, 88, 65, 91, 79};
    const int n = 5;
    print("original              : ", marks, n);

```

Output

```
original              : 72 88 65 91 79
after highest()+=2    : 72 88 65 93 79
after highest()=95    : 72 88 65 95 79
after copy += 2       : 72 88 65 95 79
const reference reads: 95
```

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp
#include <iostream>

int goodReference() {
    int x = 50;
    return x;
}

int main() {
    std::cout << "Calling goodReference()...\n";
    int r = goodReference();
    std::cout << "Returned value: " << r << '\n';

    return 0;
}
```

Output

```
Calling goodReference()...
Returned value: 50
```

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp
#include <iostream>
#include <vector>

int highestIndex(const int m[], int n) {
    int idx = 0;
    for (int i = 1; i < n; ++i) if (m[i] > m[idx]) idx = i;
    return idx;
}
int* highestPtr(int m[], int n) {
    int* p = &m[0];
    for (int i = 1; i < n; ++i) if (m[i] > *p) p = &m[i];
    return p;
}
int& highestRef(int m[], int n) { return *highestPtr(m, n); }

int main() {
    int marks[] = {72, 88, 65, 91, 79};
    marks[highestIndex(marks, 5)] += 2;
    *highestPtr(marks, 5) += 2;
    highestRef(marks, 5) += 2;
    for (int x : marks) std::cout << x << ' ';
    std::cout << "\n(91 + 2 + 2 + 2 = 97 expected)\n";


    std::vector<int> v(marks, marks + 5);
    int idx = highestIndex(v.data(), (int)v.size());
    v.insert(v.end(), 1000, 0);
    std::cout << "index still valid after growth: v[" << idx
<< "] = " << v[idx] << '\n';
}
```

Output

```
72 88 65 97 79
(91 + 2 + 2 + 2 = 97 expected)
index still valid after growth: v[3] = 97
```

Q3

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp
#include <iostream>

double charge(double weight)
    { return 50 + 20 * weight; }
double charge(double weight, bool express)
    { return 50 + 20 * weight + (express ? 30 : 0); }
double charge(double weight, bool express = false, bool
insured = false)

    { return 50 + 20 * weight + (express ? 30 : 0) + (insured
? 10 : 0); }

int main() {
    std::cout << charge(2.0) << '\n';
    std::cout << charge(2.0, true) << '\n';
}
```

Output

```
   11 |     std::cout << charge(2.0) << '\n';          //
ambiguous: overload 1 vs overload 3
      |                   ~~~~~~^~~~~
/tmp/u6fY8dwJ8D-OWGIPivp/main.cpp:5:8: note: candidate: 'double
charge(double)'
    5 | double charge(double weight)
      |        ^~~~~~
/tmp/u6fY8dwJ8D-OWGIPivp/main.cpp:7:8: note: candidate: 'double
charge(double, bool, bool)'
    7 | double charge(double weight, bool express = false, bool
insured = false)
      |        ^~~~~~
ERROR!
/tmp/u6fY8dwJ8D-OWGIPivp/main.cpp:12:24: error: call of
overloaded 'charge(double, bool)' is ambiguous
   12 |     std::cout << charge(2.0, true) << '\n';    //
ambiguous: overload 2 vs overload 3
      |                   ~~~~~~^~~~~~~~~~
/tmp/u6fY8dwJ8D-OWGIPivp/main.cpp:6:8: note: candidate: 'double
charge(double, bool)'
    6 | double charge(double weight, bool express)
      |        ^~~~~~
/tmp/u6fY8dwJ8D-OWGIPivp/main.cpp:7:8: note: candidate: 'double
charge(double, bool, bool)'
    7 | double charge(double weight, bool express = false, bool
insured = false)
      |        ^~~~~~


=== Code Exited With Errors ===
```

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp

#include <iostream>
#include <iomanip>

struct Parcel {
    double kg;
    bool express = false;
    bool insured = false;
};

static double price(double kg, bool express, bool insured) {
    double base = 40 + 20 * kg;
    double total = base;
    if (express) total += 30;
    if (insured) total += 0.02 * base;
    return total;
}

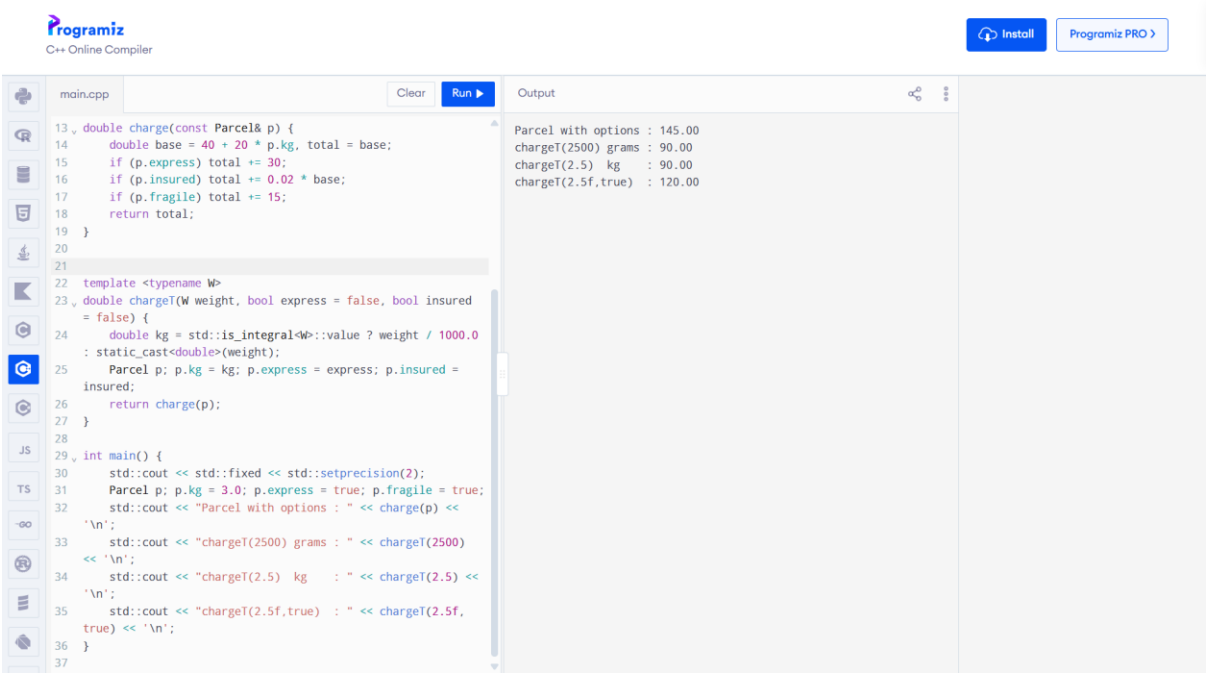
double charge(int grams, bool express = false, bool insured
= false) {
    std::cout << "[int grams ] ";
    return price(grams / 1000.0, express, insured);
}

double charge(double kg, bool express = false, bool insured
= false) {
    std::cout << "[double kg ] ";
    return price(kg, express, insured);
}
double charge(const Parcel& p) {
    std::cout << "[Parcel    ] ";
    return price(p.kg, p.express, p.insured);
```

Output

```
charge(2)            = [int grams ] 40.04
charge(2.0f)         = [double kg ] 80.00
charge(2.5)          = [double kg ] 90.00
charge(500, true)    = [int grams ] 80.00
charge(1.5,true,true)= [double kg ] 101.40
charge(p)            = [Parcel    ] 130.00
```

DHANUSH KUMAR R
192411215
DSA0120-OPP C++



Q4

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp
#include <iostream>
#include <iomanip>

struct Transaction {
    char   type;
    double amount;
};

class SavingsAccount {
    static const int MAX = 10;
    int         accNo;
    double      balance;
    Transaction last[MAX];
    int         count;

    void record(char type, double amount);
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), count(0) {}


    void deposit(double amount) {
        if (amount <= 0) return;
        balance += amount;
        record('D', amount);
    }
    bool withdraw(double amount);
    void miniStatement() const;
    double getBalance() const { return balance; }
};

```

Output

```
---- Mini statement, A/c 1001 ----
 1. Withdrawal    300.00
 2. Deposit      1000.00
 3. Deposit      1250.00
 4. Withdrawal    600.00
 5. Deposit      1750.00
 6. Deposit      2000.00
 7. Withdrawal    900.00
 8. Deposit      2500.00
 9. Deposit      2750.00
10. Withdrawal   1200.00
Balance: 14000.00

Overdraw attempt (should fail): false
```

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp
#include <iostream>
#include <deque>
#include <iomanip>

struct Transaction { char type; double amount; };

class SavingsAccount {
    static const std::size_t MAX = 10;
    int accNo; double balance; std::deque<Transaction> last;
    void record(char t, double a) {
        if (last.size() == MAX) last.pop_front();
        last.push_back({t, a});
    }
public:
    SavingsAccount(int n, double b) : accNo(n), balance(b) {}
    void deposit(double a) { balance += a; record('D', a); }
    bool withdraw(double a) { if (a > balance) return false; balance -= a; record('W', a); return true; }
    void miniStatement() const {
        std::cout << "---- Mini statement (deque), A/c " << accNo << " ----\n";
        int i = 1;
        for (const auto& t : last)
            std::cout << std::setw(2) << i++ << ". " << (t.type == 'D' ? "Deposit    " : "Withdrawal")
                      << std::setw(10) << std::fixed << std::setprecision(2) << t.amount << '\n';
        std::cout << "Balance: " << balance << "\n\n";
    }
};

```

Output

```
---- Mini statement (deque), A/c 1001 ----
 1. Withdrawal    300.00
 2. Deposit      1000.00
 3. Deposit      1250.00
 4. Withdrawal    600.00
 5. Deposit      1750.00
 6. Deposit      2000.00
 7. Withdrawal    900.00
 8. Deposit      2500.00
 9. Deposit      2750.00
10. Withdrawal   1200.00
Balance: 14000.00
```

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

```cpp

#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };

class SavingsAccount {
    static const int MAX = 10;
    int         accNo;
    double      balance;
    Transaction last[MAX];
    int         start;
    int         count;

    void record(char type, double amount) {
        if (count < MAX) {
            last[(start + count) % MAX] = {type, amount};
            ++count;
        } else {
            last[start] = {type, amount};
            start = (start + 1) % MAX;
        }
    }
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), start(0), count(0) {}
    void deposit(double a) { if (a <= 0) return; balance += a; record('D', a); }
    bool withdraw(double a) { if (a <= 0 || a > balance) return false; balance -= a; record('W', a); return true; }
    void miniStatement() const {
        std::cout << "---- Mini statement (circular), A/c "
```

Output

```
---- Mini statement (circular), A/c 1001 ----
 1. Withdrawal    300.00
 2. Deposit      1000.00
 3. Deposit      1250.00
 4. Withdrawal    600.00
 5. Deposit      1750.00
 6. Deposit      2000.00
 7. Withdrawal    900.00
 8. Deposit      2500.00
 9. Deposit      2750.00
10. Withdrawal   1200.00
Balance: 14000.00
```

Q5

Programiz
C++ Online Compiler

Install | Programiz PRO >

main.cpp | Clear | Run

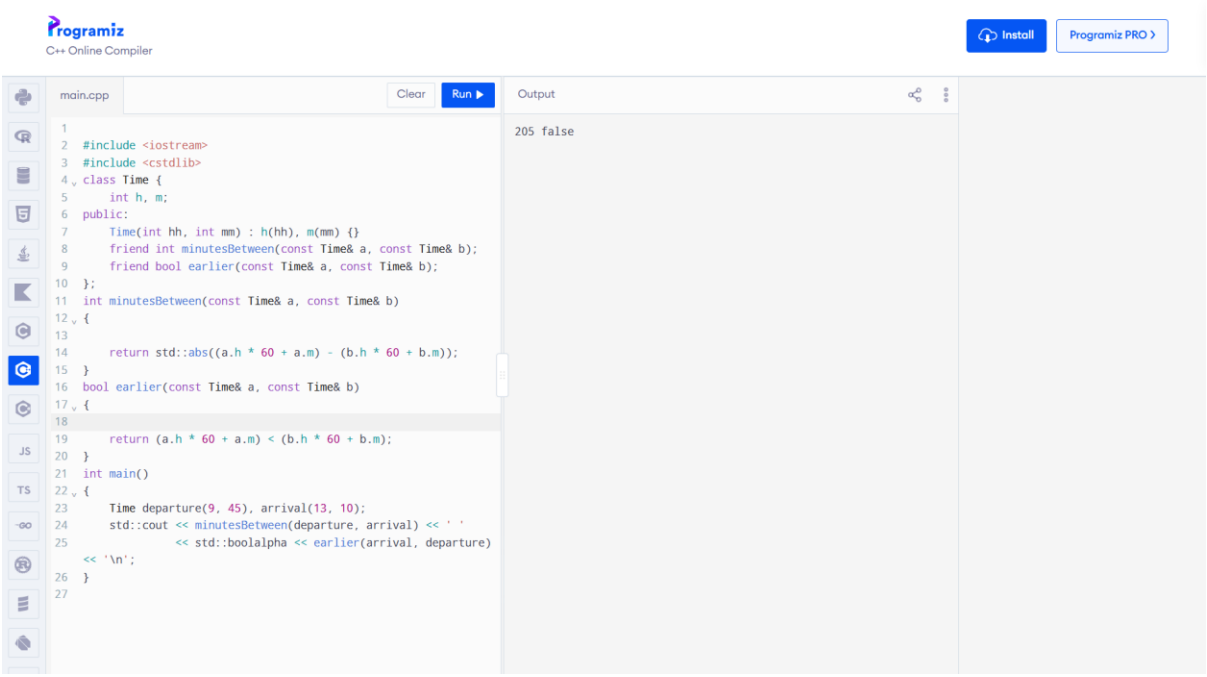
```cpp
#include <iostream>
#include <string>
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }
    static int count()
    {
        // TODO 2
        return issued;
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};
int BusTicket::issued = 0;
int main()
```

Output

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

Q6

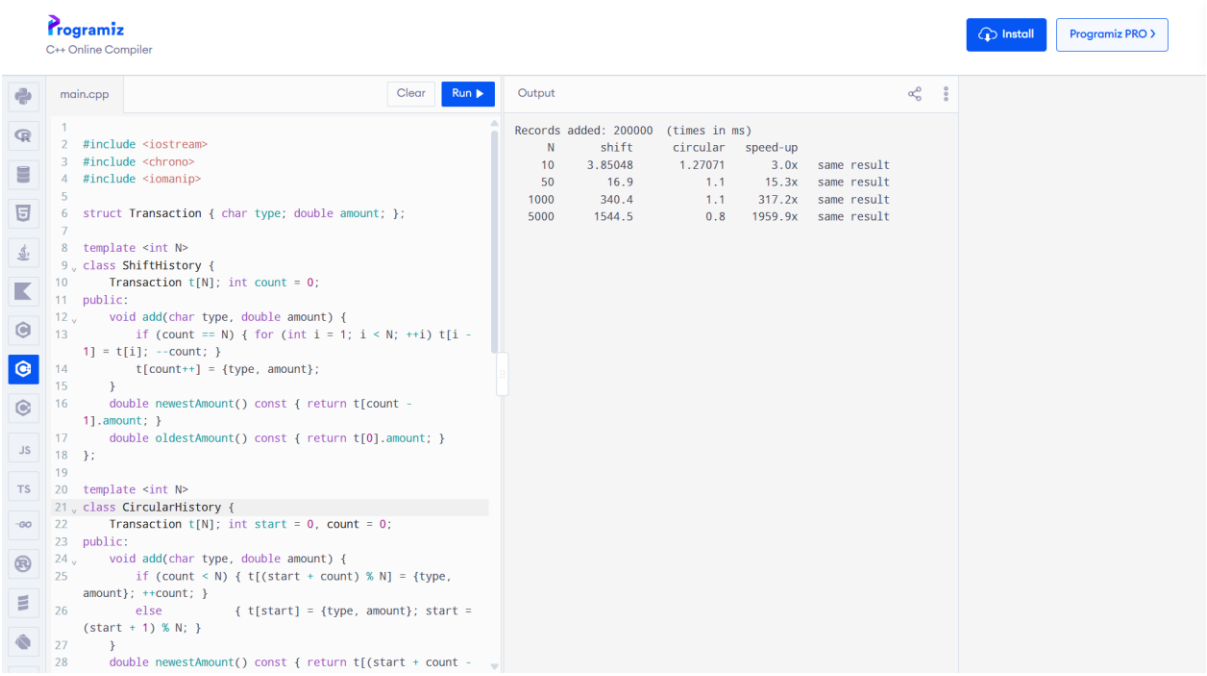



```cpp
#include <iostream>
#include <cstdlib>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{

    return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m));
}
bool earlier(const Time& a, const Time& b)
{

    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}
int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure)
<< '\n';
}
```

Q7




```cpp
#include <iostream>
#include <chrono>
#include <iomanip>

struct Transaction { char type; double amount; };

template <int N>
class ShiftHistory {
    Transaction t[N]; int count = 0;
public:
    void add(char type, double amount) {
        if (count == N) { for (int i = 1; i < N; ++i) t[i -
1] = t[i]; --count; }
        t[count++] = {type, amount};
    }
    double newestAmount() const { return t[count -
1].amount; }
    double oldestAmount() const { return t[0].amount; }
};

template <int N>
class CircularHistory {
    Transaction t[N]; int start = 0, count = 0;
public:
    void add(char type, double amount) {
        if (count < N) { t[(start + count) % N] = {type,
amount}; ++count; }
        else           { t[start] = {type, amount}; start =
(start + 1) % N; }
    }
    double newestAmount() const { return t[(start + count -
```

DHANUSH KUMAR R
192411215
DSA0120-OPP C++

REPORT NOTES

Chosen program: Q4 SavingsAccount (fixed array, shift on overflow).

A. Diagnosis - add() costs O(N) once the array is full (loop shifting N-1 elements); with N = 10 it is negligible, with N = 50 / 1000 / 5000 it grows linearly. Cause: oldest entry is always at index 0, so every insertion moves all others.

B. Objective - make add() time independent of N (O(1)); target: at N = 1000, at least 20x faster than shifting for 200,000 transactions, with identical mini-statement output.

C. Redesign - keep the private fixed array, add `start` index; new entry overwrites `last[start]` when full; print from `start` using `(start+i) % N`. No extra memory except one int. Trade-off: slightly more complex printing/index arithmetic; elements no longer contiguous in logical order.

D. Prediction - time per add stays flat as N grows for circular; shift time grows about linearly with N (e.g. N=1000 ~ 100x slower than N=10).

E. Validation - run `redesign_benchmark.cpp` (-O2) and paste the table; also compare account_shift vs account_circular outputs (identical).

F. Defence - objective met if circular time is flat. Original may still be preferable for N <= 10 (simple, contiguous, trivially readable) or when the statement must be printed/copied as one contiguous block very often